SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I Mukundhan VR *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

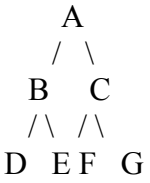
Mukundhan

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



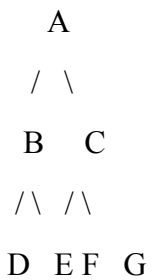
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

SIMATS ENGINEERING

ASSESSMENT TOOL 3 – DRY RUN EVALUATION

Course: Artificial Intelligence | Course Outcome: CO2

Name / Reg. No.: mukundhan/1972260015

1. Breadth-First Search (BFS)

BFS visits nodes level by level. Starting at A, it uses a queue (FIFO: first in, first out). Assuming the children are visited from left to right, the graph is A → B, C; B → D, E; C → F, G.

Iteration	Queue before visit	Visited node	Queue after visit
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]
5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[]

BFS traversal order: A, B, C, D, E, F, G.

Queue after each iteration: [B, C] → [C, D, E] → [D, E, F, G] → [E, F, G] → [F, G] → [G] → [].

2. Depth-First Search (DFS)

DFS follows one branch as far as possible before backtracking. Taking the graph from left to right, start at A, then go to B, D, backtrack to B, visit E, backtrack to A, then visit C, F, and G.

DFS traversal order: A, B, D, E, C, F, G.

Stack idea: A → B → D; backtrack to B → E; backtrack to A → C → F; backtrack to C → G.

3. 8-Puzzle – BFS Dry Run

Initial state:

1	3	6
5	0	2
4	7	8

Goal state:

1	2	3
4	5	6
7	8	0

Important note: with the standard 8-puzzle rules (move the blank one square up, down, left, or right), this initial arrangement is not solvable to the stated goal. Its inversion count is 11 (odd), while the goal has an even inversion count. Therefore, a valid BFS run cannot reach the goal from the given initial state. The table below shows the first five queue expansions instead of inventing a goal-reaching path.

Iteration	Node expanded	Queue before expansion	Queue after expansion
1	Initial state S0	[S0]	[S1, S2, S3]
2	First generated state S1	[S1, S2, S3]	[S2, S3, S4, S5]
3	Second generated state S2	[S2, S3, S4, S5]	[S3, S4, S5, S6, S7]
4	Third generated state S3	[S3, S4, S5, S6, S7]	[S4, S5, S6, S7, S8, S9]
5	Fourth generated state S4	[S4, S5, S6, S7, S8, S9]	[S5, S6, S7, S8, S9, S10, S11]

Node expanded in each iteration: BFS expands the state at the front of the queue. The table labels the states by their queue order; the exact state order depends on the chosen order for generating legal moves.

Total states: The number of states generated before reaching the goal cannot be reported as a finite goal-reaching count for this initial state, because the goal is unreachable. BFS will exhaust the reachable component if allowed to run fully.

Solution path: No solution path exists for the given initial and goal states under standard 8-puzzle moves. The initial state has 11 inversions (ignoring 0), which is odd; the goal has 0 inversions, which is even. The parity differs, so the puzzle is unsolvable.

Why BFS finds the shortest solution in an unweighted puzzle: BFS explores states in increasing order of depth. Each move has equal cost, so the first time BFS reaches a goal state, it uses the fewest possible moves.

Before submission, review the dry run and make sure you understand the queue updates. The 8-puzzle question appears to contain an initial state that cannot reach the stated goal, so it may be worth confirming the arrangement with your instructor.